3. 梯度是怎么算出来的？(Section I.C: Gradient Back-Propagation)

在理解了模型通过“梯度下降”来学习（调水温）之后，随之而来的一个核心问题是：**面对网络中成千上万个参数（旋钮），我们怎么知道每个旋钮该往哪边拧、拧多少？**（即如何计算梯度 $$\frac{\partial E(W)}{\partial W}$$）

这就引出了深度学习的“心脏”算法——**反向传播（Back-Propagation, 简称 BP）**。

3.1 避坑指南：反向传播到底在调整什么？

- ⚠ **常见误区**：认为反向传播是为了调整误差函数 $$E(W)$$。
- ☑ **正确理解**：$$E(W)$$ 是一把“固定的尺子”（比如预测值减去真实值的平方），它的计算公式是提前写死、绝对不变的。**反向传播绝不是去修改这把尺子，而是为了计算出如何调整参数 $$W$$。**

3.2 深度学习训练的“流水线闭环”（核心逻辑）

我们可以把整个训练过程想象成一个公司运作的**“流水线闭环”**，由四个部门紧密配合：

1. **前向传播 (Forward Pass) —— “员工干活”**
   - 数据从网络的输入层开始，经过一层层的非线性变换（主管、经理审批），最终在输出层给出一个预测结果。
2. **误差函数 $$E(W)$$ —— “老板量 KPI”**
   - 用预测结果对比真实答案（标签），通过固定的公式算出一个总误差（Loss）。（老板用固定的 KPI 尺子量了一下，发现公司这个月亏钱了）。
3. **反向传播 (Backward Pass) —— “HR 部门层层追责”**
   - 利用微积分的**链式法则（Chain Rule）**，从输出层（尾部）开始，一层一层往回倒推。
   - **它的唯一目的就是算账（求导数/算梯度）**：算出每一个参数 $$W$$ 对最终误差 $$E$$ 的“责任大小”，也就是得到梯度 $$\frac{\partial E(W)}{\partial W}$$。（HR 部门算出每个员工该为这次亏损背多少锅）。
4. **梯度下降 (Gradient Descent/SGD) —— “财务部动手扣钱”**
   - 拿到反向传播算好的梯度（账本）后，套用公式：$$W_{new} = W_{old} - \epsilon \frac{\partial E(W)}{\partial W}$$。
   - **真正动手去更新所有的参数 $$W$$。**（财务部根据 HR 的账本和固定的惩罚力度 $$\epsilon$$，正式扣除或增加每个员工的工资）。

3.3 一句话总结

- **$$E(W)$$** 是**目标**（尺子不变）。
- **反向传播 (BP)** 是**找方向**（算梯度，找出谁该负责）。
- **梯度下降 (SGD)** 是**迈步子**（动手调参数 $$W$$）。
- 它们合伙的最终目的，是为了让下一次前向传播时，算出来的误差 $$E(W)$$ 变得更小！